Maximizing Submodular Functions

Paul Sautner

April 2023

1 Introduction

Submodular functions are found in a number of different applications. They exhibit a diminishing returns property that arises naturally in many problems. In this paper, we will look at the problem of maximizing nonnegative, monotonous, submodular functions. After giving definitions of these terms in Section 2, we will first consider the case of a cardinality constraint in Section 3. For this constraint, we will present two algorithms and prove upper bounds for the value difference of the returned solution and an optimal solution. After that, we will look at a more complex type of constraint, the matroid constraint, in Section 4. We will present modified versions of the previously presented algorithms and again show their approximation guarantees. Finally, we will present two more involved algorithms that achieve even better results for this type of constraint.´ A JavaScript implementation and demo of the presented algorithms can be found at `https://github.com/Pausau/SubmodularFunctions`.

2 Submodular functions

Let E be a finite set and $\mathcal{P}(E)$ the set of all subsets of E. A function $f : \mathcal{P}(E) \to \mathbb{R}$ is called **submodular** if for $S, T \subseteq E$ it holds that

$$f(S) + f(T) \geq f(S \cup T) + f(S \cap T).$$

If equality holds in the above equation, we call f **modular**.

There is another, sometimes more convenient, characterization of submodular functions which we will give next.
Let $e \in E$ and $S \subseteq E$. For a function $f : \mathcal{P}(E) \to \mathbb{R}$, we define the **gain** of adding e to S as

$$\Delta_f(e|S) = f(S \cup \{e\}) - f(S).$$

The following two statements are equivalent:
1) For every $A \subseteq B \subseteq E$ and $e \in E$ it holds that

$$\Delta_f(e|A) \geq \Delta_f(e|B),$$

2) f is submodular.

Proof.
1) $\implies$ 2)
After choosing an ordering for the elements of $S \cup T = \{e_1, ..., e_{|S \cup T|}\}$, we can write $f(S \cup T)$ as a telescoping sum:

$$f(S \cup T) = f(\emptyset) + \sum_{i=1}^{|S \cup T|} \Delta_f(e_i, \{e_1, ..., e_{i-1}\}).$$

We can choose the ordering in such a way that $e_i \in S \cap T$ for $i \leq |S \cap T|$, $e_i \in S \setminus T$ for $|S \cap T| \leq i \leq |S|$ and $e_i \in T \setminus S$ for $|S| \leq i \leq |T \cup S|$. This way, we can write the above sum as

$$\begin{aligned} f(S \cup T) &= f(\emptyset) + \sum_{i=1}^{|S|} \Delta_f(e_i, \{e_1, ..., e_{i-1}\}) + \sum_{i=|S|}^{|S \cup T|} \Delta_f(e_i, \{e_1, ..., e_{i-1}\}) \\ &= f(S) + \sum_{i=|S|}^{|S \cup T|} \Delta_f(e_i, \{e_1, ..., e_{i-1}\}). \end{aligned}$$

Using this representation, we can use the assumption to bound the term:

$$\begin{aligned} f(S \cup T) &\leq f(S) + \sum_{i=|S|}^{|S \cup T|} \Delta_f(e_i, \{e_1, ..., e_{i-1}\} \setminus (S \setminus T)) \\ &= f(S) + \sum_{i=|S|}^{|S \cup T|} \Delta_f(e_i, \{e_1, ..., e_{|S \cap T|}\} \cup \{e_{|S|}, ..., e_{i-1}\}). \end{aligned}$$

The last sum is again a telescoping sum. We get

$$f(S \cup T) \leq f(S) + (f(T) - f(S \cap T)).$$

Rearranging the terms yields the desired result:

$$f(S \cup T) + f(S \cap T) \leq f(S) + f(T).$$

2) $\implies$ 1)
Let $A \subseteq B \subseteq E$. Set $S := A \cup \{e\}$, $T := B$. Then 2) states

$$f(A \cup \{e\}) + f(B) \geq f(B \cup \{e\}) + f(A),$$

which is equivalent to

$$\Delta_f(e|A) \geq \Delta_f(e|B).$$

□

We will also limit ourselves to *monotonous* functions. A set function f is called **monotonous** if for $A \subseteq B \subseteq E$ it holds that:

$$f(B) \geq f(A),$$

or equivalently,

$$\Delta_f(e|A) \geq 0 \ \forall e \in E.$$

Additionally, we will assume that f is **nonnegative**, i.e. $f(S) \geq 0$ for all $S \subseteq E$.

We will now concern ourselves with the problem of maximizing a given submodular, monotonous, nonnegative function:

$$\max_{S \subseteq E} f(S).$$

We will measure the complexity of an algorithm in terms of function evaluations of f. When we evaluate f, we say that we call the **value oracle**.
In the unconstrained case, the problem is trivial because of monotonicity: the set E always maximizes f. Therefore we will now look at some constraints that arise in different applications.

3 Cardinality constraint

One of the simplest constraints is the cardinality constraint, where we restrict the possible solutions to subsets no larger than a certain size:

$$\max_{S \subseteq E} f(S) \text{ such that } |S| \leq r.$$

The problem of finding an optimal solution for this constraint is already NP-hard [3]. We will be concerned with efficient algorithms whose returned solutions are provably close to the optimal solution.

A simple approach for this problem would be to start with the empty set and then iteratively add the next-best element to the set until we have added r elements. This approach is called the **greedy algorithm**.
The greedy algorithm has a running time of $O(nr)$ since in every iteration of the loop, we need to evaluate the function n times, and there are r iterations.
It gives a $(1-1/e)$-approximation, meaning that if O is an optimal solution and A is the solution returned by the algorithm, it holds that

$$f(A) \geq (1 - 1/e) f(O),$$

where $e = \lim_{n \to \infty}(1 + 1/n)^n$ is Euler's number.

Algorithm 1 Greedy Algorithm

Input: $f : \mathcal{P}(E) \to \mathbb{R}_+, r \in \{1, ..., n-1\}$.
Output: A set $S \subset E$ satisfying $|S| \leq r$.

```
S ← ∅
while |S| < r do
    maxGain ← 0
    for e ∈ E do
        if Δ_f(e|S) > maxGain then
            nextElement ← e
            maxGain ← Δ_f(e|S)
        end if
    end for
    S ← S ∪ {nextElement}
end while
return S
```

Proof.
Let O be an optimal solution, let $S_i = \{a_1, ..., a_i\}$ be the selected set after i steps and let $\{o_1, ..., o_{|O \setminus S_i|}\}$ be the elements of $O \setminus S_i$. First, observe that by submodularity and monotonicity, we have

$$\begin{aligned}\sum_{k=1}^{|O\setminus S_i|} \Delta_f(o_k|S_i) &\geq \sum_{k=1}^{|O\setminus S_i|} \Delta_f(o_k|S_i \cup \{x_1, ..., x_{k-1}\}) \\ &= f(O \cup S_i) - f(S_i) \\ &\geq f(O) - f(S_i).\end{aligned}$$

Because the next element chosen by the greedy algorithm a_{i+1} is the element with the maximum gain, the gain of adding it to the current set is at least as big as the average gain of adding any element of $O \setminus S$ to the current set:

$$\Delta_f(a_{i+1}|S_i) \geq \frac{1}{|O \setminus S|} \sum_{k=1}^{|O\setminus S_i|} \Delta_f(o_k|S_i).$$

Since $|O \setminus S_i| \geq |O| \geq r$, this implies

$$\Delta_f(a_{i+1}|S_i) \geq \frac{1}{r} \sum_{k=1}^{|O\setminus S_i|} \Delta_f(o_k|S_i).$$

Combining this with the observation above, we have

$$f(S_{i+1}) - f(S_i) = \Delta_f(a_{i+1}|S_i) \geq \frac{1}{r}(f(O) - f(S_i))$$

$$\iff f(S_{i+1}) \geq \frac{1}{r} f(O) + (1 - \frac{1}{r}) f(S_i).$$

Inductively, we get

$$\begin{aligned} f(S_{i+1}) &\geq (1-\frac{1}{r})^{i+1} f(S_0) + \sum_{j=0}^{i} (1-\frac{1}{r})^j (\frac{1}{r} f(O)) \\ &= (1-\frac{1}{r})^{i+1} f(\emptyset) + (\frac{1}{r} f(O)) \frac{1-(1-\frac{1}{r})^{i+1}}{1-(1-\frac{1}{r})} \\ &= (1-\frac{1}{r})^{i+1} f(\emptyset) + f(O)(1-(1-\frac{1}{r})^{i+1}) \\ &\geq f(O)(1-(1-\frac{1}{r})^{i+1}), \end{aligned}$$

where we used the nonnegativy of f in the last line.
Looking at S_r, we have

$$\begin{aligned} f(S_r) &\geq (1-(1-\frac{1}{r})^r) f(O) \\ &\geq (1-1/e) f(O). \end{aligned}$$

□

One simple modification of the greedy algorithm that is often used in practice uses the submodularity of f to make a potentially smaller number of oracle calls. The idea is to maintain a list of upper bounds of the gains of all elements. In the first iteration of the greedy algorithm, we can save the gain of adding each element to the empty set in a list and order that list highest to lowest. Because of submodularity, the gain of a particular element can only decrease after we add an element to our current set. This means that the values of the list are upper bounds on the gains of the elements in the next iteration. Just like in the greedy algorithm, we will add the element with the highest gain, that is the first element in our list, to our current solution. If, in the second iteration, the gain of adding the second element to the current in the list is greater than the gain of adding the third element to the empty set, then the second element is guaranteed to be the element with maximum gain. If the updated gain is less than the upper bound of the next element, we will just insert the updated upper bound in our list. We can continue in this way and hope that we will not need to make an oracle call for each element in each iteration. This modified algorithm is called the **lazy greedy algorithm**.
Since this algorithm adds the same elements as the greedy algorithm, it achieves the same approximation. Depending on the problem, we might be unlucky and will still need the same number of oracle calls. But we had to perform all the (in this case unnecessary) list operations of sorting and inserting.

We now present a third algorithm whose running time does not depend on r which might be beneficial, depending on the application. Later, we will use the idea from this algorithm for the case of a more complicated constraint.
The idea is to have a threshold that is iteratively decreased. In each iteration

of the algorithm, we will add all elements whose gain is above the threshold. At the end of each iteration, we will multiply the threshold by a fixed factor $(1-\epsilon)$, where $\epsilon \in (0,1)$ is a parameter that we can choose. We continue this procedure until the threshold is so small that the current solution is guaranteed to be a $(1-1/e-\epsilon)$-approximation.

Algorithm 2 Exponential Thresholding Algorithm

Input: $f : \mathcal{P}(E) \to \mathbb{R}_+, r \in \{1, ..., n-1\}$.
Output: A set $S \subset E$ satisfying $|S| \leq r$.

```
S ← ∅
d ← max_{e∈E} f({e})
for (w = d; w ≥ (ϵ/n)d; w ← w(1 − ϵ)) do
    for e ∈ E do
        if |S ∪ {e}| ≤ r and Δ_f(e|S) ≥ w then
            S ← S ∪ {e}
        end if
    end for
end for
return S
```

Since this algorithm consists of nested loops, the number of oracle calls is equal to the product of iterations of each loop: $(\log(d) - \log(\frac{\epsilon}{n}d))/\log(1-\epsilon) \approx \frac{1}{\epsilon}\log(\frac{n}{\epsilon})$ for the outer loop and n for the inner loop, giving a total running time of $O(\frac{n}{\epsilon}\log(\frac{n}{\epsilon}))$.
It achieves a $(1-e-\epsilon)$-approximation.

Proof.
We will show that the gain of each element that is added to the set is lower bounded. Let a be the next element chosen and w be the current threshold. We have the following inequalities:
(1) $\Delta_f(a|S) \geq w$
(2) $\Delta_f(e|S) \leq w/(1-\epsilon)$ for all $e \in E$.
Combining them yields

$$\Delta_f(a|S) \geq w \geq (1-\epsilon)\Delta_f(e|S) \text{ for all } e \in E \text{ (in particular for } e \in O \setminus S).$$

Averaging over the elements in $O \setminus S$ gives

$$\Delta_f(a|S) \geq \tfrac{1-\epsilon}{|O\setminus S|}\textstyle\sum_{e\in O\setminus S}\Delta_f(e|S) \geq \tfrac{1-\epsilon}{r}\textstyle\sum_{e\in O\setminus S}\Delta_f(e|S).$$

From here, we can follow the exact same reasoning as in the case of the greedy algorithm to complete the proof. □

4 Matroid constraint

A more complex type of constraint that arises in many applications is that of matroid constraints. We will first need to define what a matroid is.

Let E be a finite set. A **matroid** is a pair $(E, \mathcal{I})$, where $\mathcal{I}$ is a family of subsets of E (called the **independent sets**) satisfying the following properties:

- $\emptyset \in \mathcal{I}$,
- $A \in \mathcal{I} \wedge B \subseteq A \implies B \in \mathcal{I}$,
- $A, B \in \mathcal{I} \wedge |A| > |B| \implies \exists x \in A : (B \cup \{x\}) \in \mathcal{I}$.

Example (Partition Matroid)
Let $W_1, ..., W_i$ be a partition of E and let $r_1, ..., r_i \in \mathbb{N}$. Now define

$$A \subseteq E \text{ independent } :\iff |A \cap W_k| \leq r_k \; \forall k \in \{1, ..., i\}.$$

The independent sets clearly satisfy the properties above. This particular type of matroid is called a **partition matroid**.

A maximal independent set, i.e. a set that becomes dependent after adding any $e \in E$ to it, is called a **basis** of the matroid. It is a well-known fact that all bases of a matroid have the same cardinality r, which we call the **rank** of the matroid.

The optimization problem will now consist of finding an independent set that maximizes f:

$$\max_{S \subseteq E} f(S) \text{ such that } S \in \mathcal{I}.$$

We will assume that there is an **independence oracle** that we can call to tell us whether a given set is independent.

We can modify the greedy algorithm from the last section to work for the new constraint.
This algorithm makes $O(n^2)$ oracle calls. It achieves a 1/2-approximation.

Proof.
Let O be an optimal solution and let S_i be the set selected after i steps. Note that the greedy algorithm only terminates after adding r elements (making $S_r = \{a_1, ..., a_r\}$ a basis of the matroid). Writing $f(O \cup S) - f(S)$ as a telescoping

Algorithm 3 Greedy Algorithm (Matroid)

Input: $f : \mathcal{P}(E) \to \mathbb{R}_+, r \in \{1, ..., n-1\}$.
Output: A set $S \subset E$ satisfying $|S| \leq r$.

```
S ← ∅
C ← E
while C ≠ ∅ do
    maxGain ← 0
    for e ∈ E do
        if Δ_f(e|S) > maxGain then
            nextElement ← e
            maxGain ← Δ_f(e|S)
        end if
    end for
    if (S ∪ nextElement) is independent then
        S ← S ∪ {nextElement}
    end if
    C ← C \ {nextElement}
end while
return S
```

sum and using monotonicity and submodularity, we observe that

$$\begin{aligned} f(O) - f(S) &\leq f(O \cup S) - f(S) \\ &= \sum_{k=1}^{r} \Delta_f(o_k | S \cup \{o_1, ..., o_{k-1}\}) \\ &\leq \sum_{k=1}^{r} \Delta_f(o_k | S_i) \\ &\leq \sum_{k=1}^{r} \Delta_f(a_k | S_i) = f(S). \end{aligned}$$

Going from line 3 to 4, we used the fact that the greedy algorithm always chooses the element of maximum gain. Rearranging, we obtain the result:

$$f(O) \leq 2f(S) \iff f(S) \geq \frac{1}{2} f(O).$$

□

With the same modifications as in the cardinality constraint case, this algorithm can also be turned into a "lazy" version that might reduce the number of oracle calls.

Similarly, the exponential thresholding algorithm can also be adapted for the matroid constraint.
It makes $O(\frac{n}{\epsilon} \log(\frac{n}{\epsilon}))$ oracle calls and achieves a $(1/2 - \epsilon)$-approximation.

Algorithm 4 Exponential Thresholding Algorithm (Matroid)

Input: $f : \mathcal{P}(E) \to \mathbb{R}_+$, Matroid $(E, \mathcal{I})$.
Output: A set $S \subset E$ satisfying $S \in \mathcal{I}$.

```
S ← ∅
d ← max_{e∈E} f({e})
for (w = d; w ≥ (ϵ/n)d; w ← w(1 − ϵ)) do
    for e ∈ E do
        if (S ∪ {e}) ∈ I and Δ_f(e|S) ≥ w then
            S ← S ∪ {e}
        end if
    end for
end for
return S
```

Proof.
The proof is very similar to the one of the greedy algorithm.
Let O be an optimal solution and let $S_i = \{a_1, ..., a_i\}$ be the set selected after i steps. We have

$$\begin{aligned} f(O) - f(S) &\leq f(O \cup S) - f(S) \\ &= \sum_{k=1}^{r} \Delta_f(o_k | S \cup \{o_1, ..., o_{k-1}\}) \\ &\leq \sum_{k=1}^{r} \Delta_f(o_k | S_i) \\ &\leq \sum_{k=1}^{r} \Delta_f(a_k | S_i)/(1-\epsilon) = f(S)/(1-\epsilon). \end{aligned}$$

Here we used the fact that $\Delta_f(a_i|S_i)/(1-e) \geq \Delta_f(a_i|S_i)(e)$ for all $e \in E$ (which we established in the proof for the exponential thresholding algorithm in the cardinality constraint case).
Rearranging, we get

$$\begin{aligned} f(O)(1-\epsilon) \leq f(S) + f(S)(1-\epsilon) &\iff f(S)(2-\epsilon) \geq f(O)(1-\epsilon) \\ &\iff f(S) \geq \frac{1-\epsilon}{2-\epsilon} f(O) \geq (\frac{1}{2} - \epsilon) f(O). \end{aligned}$$

The very last inequality is easily verified by looking at the derivative of $g(\epsilon) = \frac{1-\epsilon}{2-\epsilon}$ and its value at $\epsilon = 0$. □

Compared to the cardinality constraint case, these algorithms give worse approximations. Vondrak found a probabilistic algorithm that gives a $(1-1/e)$-approximation even for a matroid constraint, called the **continuous greedy algorithm** [4]. The idea is to work with random sets that contain each element

$e \in E$ with a certain probability. These random sets can be represented by a vector $p_t \in [0,1]^{|E|}$. The algorithm starts with a random set with probability 0 for each element, that is $p_0 = 0$, and then iteratively updates the probabilities as to maximize the expected value of f. The updates of p_t are constrained in such a way, that only the probabilities of members of one independent set can be updated in one step. This makes it possible to, in the end, round each entry of p_t to either 0 or 1 and thereby achieve an $(1 - 1/e)$ approximation, which they prove in [4].

Before we continue, we should introduce some notation. For a vector of probabilities p_t, let $R(p_t)$ denote a set that contains each element $e \in E$ with the corresponding probability. We define the **expected gain** of adding $e \in E$ to the random set $R(p_t)$ to be $\mathbb{E}[\Delta(e|R(p_t)]$.

Now we can give a more detailed description of the continuous greedy algorithm: the algorithm starts at time $t = 0$. In each iteration, the expected gain of adding each element to the current random set is estimated by averaging n^5 samples for each element. After that, an independent set, whose sum of expected gains is maximal, is selected. This can be done efficiently using the greedy algorithm which is known to return an optimal solution. After this independent set is chosen, the probability of the current random set containing each element of this independent set is increased by δ. At the end of each iteration, the time t is incremented by δ. This procedure continues until $t \geq 1$. At this point, we still need to round the entries of the final probability vector p_1 to integer values. This can be done using a technique called *pipage rounding* which was first introduced in [1].

If we choose $\delta = O(1/n)$, the running time of this algorithm is $O(n^7)$ calls to the oracle. This may be prohibitively expensive even for small problems.

Algorithm 5 Continuous Greedy

Input: $f : \mathcal{P}(E) \to \mathbb{R}_+$, Matroid $(E, \mathcal{I})$.
Output: A set $S \subseteq E$ satisfying $S \in \mathcal{I}$.

```
for e ∈ E do
    p(0, e) ← 0
end for
for (t = 0; t < 1; t ← t + δ) do
    for e ∈ E do
        w(t, e) ← estimate of E[Δ(e|R(p(t, ·))] by taking the average of n^5
samples.
    end for
    I(t) ← maximum-weight independent set, according to w(t, e).
    for e ∈ I(t) do
        p(t + δ, e) ← p(t, e) + δ
    end for
end for
return PipageRounding(p(1, ·))
```

Next we present an algorithm that achieves a $(1 - 1/e - \epsilon)$-approximation and has a better running time. It was first presented in [2].

The algorithm, which is called the **accelerated continuous greedy algorithm**, modifies the continuous greedy algorithm in two ways: First, it applies the idea of exponential thresholding. Second, instead of estimating the expected gains of each element and then choosing a maximum-weight independent set, it already increases the probabilities before a maximal independent set has been selected.

Algorithm 6 Accelerated Continuous Greedy

Input: $f : \mathcal{P}(E) \to \mathbb{R}_+$, Matroid $(E, \mathcal{I})$.
Output: A set $S \subseteq E$ satisfying $S \in \mathcal{I}$.

```
for e ∈ E do
    p(0, e) ← 0
end for
for (t = 0; t < 1; t ← t + δ) do
    S ← ∅
    d ← max_{e∈E} f_S(e)
    for (w = d; w ≥ (ϵ/n)d; w ← w(1 − ϵ)) do
        for e ∈ E do
            w(t, S, e) ← estimate of E[Δ(e|R(p(t, ·)))] by taking the average of (r log n)/ϵ² samples.
            if (S ∪ {e}) ∈ I and w(t, S, e) ≥ w then
                S ← S ∪ {e}
            end if
        end for
    end for
    for e ∈ S do
        p(t + δ, e) ← p(t, e) + δ
    end for
end for
return PipageRounding(p(1, ·))
```

The running time is easy to determine since the algorithm consists of nested loops. We have $1/\delta$ iterations of the outer loop, $O(\frac{1}{\epsilon}\log(\frac{n}{\epsilon}))$ iterations in the exponential thresholding loop, n iterations of the inner-most loop, and in each iteration we take $\frac{r \log n}{\epsilon^2}$ samples. This gives a total running time of $O(\frac{nr}{\epsilon^3 \delta}\log(n)\log(\frac{n}{\epsilon}))$.

If we choose $\delta = \epsilon$, the algorithm achieves a $(1 - 1/e - O(\epsilon))$-approximation.

Proof.
Just like in the proofs for the cardinality constraint, we will proceed by first lower bounding the improvement in each iteration and then use that to prove the statement. For this setting, we will look at the improvement in one time step.
Before we start, however, we introduce some notation: for $p_t \in [0,1]^{|E|}$, we denote by $F(p_t) = \mathbb{E}[f(R_{p_t})]$ the expected value of f and for $S \subseteq E$ and we denote by $p_t + \delta S$ the updated probability vector, where the probability of each $e \in S$ was increased by δ.
Note that F is componentwise affine: If we fix all probabilities and only let the probability p_t^a of $a \in E$ be variable, we get a function $g : [0,1] \to \mathbb{R}$. Defining p_t' and $\tilde{p_t}$ to be copies of p_t where the entry corresponding to a is changed to 1 and 0 respectively, and using the law of total expectation we have:

$$g(p_t^a) = p_t^a F(p_t') + (1 - p_t^a)F(\tilde{p_t}) = F(\tilde{p_t}) + p_t^a(F(p_t') - F(\tilde{p_t})).$$

Also note that the monotonicity of f implies the monotonicity of F in the following sense:

$$p' \geq p \implies F(p') \geq F(p),$$

where $p' \geq p$ is to be understood componentwise.

We now want to lower bound the improvement at each step, i.e. $F(p_{t+\delta}) - F(p_t)$. Let $S = \{a_1, ..., a_r\}$ be the independent set that is selected at the current time step and S_i be the set of the first i elements of S. (If the exponential thresholding loop terminates before r elements were added, we just fill it up with whatever elements make it a basis.) We can write $F(p_{t+\delta}) - F(p_t)$ as a telescoping sum:

$$F(p_{t+\delta}) - F(p_t) = \sum_{i=1}^{r} F(p_t + \delta S_i) - F(p_t + \delta S_{i-1}).$$

When we compare $p_t + \delta S_{i-1}$ and $p_t + \delta S_i$, we see that only the probability $p_t^{a_i}$ of a_i changes (it is incremented by δ) and we know that F changes linearly with $p_t^{a_i}$. Hence we have

$$F(p_t + \delta S_i) - F(p_t + \delta S_{i-1}) = \delta g'(p_t^{a_i}) = \delta(F(p_t') - F(\tilde{p_t})),$$

where p_t' and $\tilde{p_t}$ are defined as above but for a_i.
Because of monotonicity, we have $F(p_t') - F(\tilde{p_t}) \geq F(p_t') - F(p_t) = \mathbb{E}[\Delta_f(a_i | R(p_t))]$, which we will establish a lower bound for next.

The *relative and additive Chernoff bound* states that if $X_1, ..., X_n$ are independent random variables such that $X_i \in [0,1]$ and $X = \frac{1}{m}\sum_{i=1}^{m} X_i$ is their average, then

$$\mathcal{P}(|X - \mathbb{E}(X)| > \alpha\mathbb{E}(X) + \beta) \leq e^{-\frac{m\alpha\beta}{2}} + e^{-\frac{m\alpha\beta}{3}}.$$

A proof of this can be found in [2].
Let O be an optimal solution. Setting $\alpha = \epsilon$, $\beta = \frac{\epsilon}{r} f(O)$ and $m = \frac{1}{\epsilon^2} r \log n$ (which is the number of samples in the algorithm), we have

$$\mathcal{P}(|X - \mathbb{E}(X)| > \epsilon(\mathbb{E}(X) + \frac{1}{r} f(O)) \leq e^{-\frac{\log n f(O)}{2}} + e^{-\frac{\log n f(O)}{3}} = n^{\frac{-f(O)}{2}} + n^{\frac{-f(O)}{3}},$$

which approaches 0 for $n \to \infty$.
A well-known result in matroid theory is the following: if B_1 and B_2 are two bases of a matroid, then there exists a bijection $\phi : B_1 \to B_2$ such that for every $b \in B_1$ it holds that $(B_1 \setminus b) \cup \phi(b)$ is a basis.
For a fixed bijection ϕ, let $O = \{o_1, ..., o_r\}$ be ordered in such a way that $o_i = \phi(a_i)$. If a_i was chosen as the next element to add to S_{i-1}, this means that we could have chosen o_i instead. But because of how the algorithm works, it must have been the case that

$$w(t, S_{i-1}, a_i) \geq (1 - \epsilon) w(t, S_{i-1}, o_i)$$

if a_i was added during the exponential thresholding.
If a_i was added after that to complete the returned set to a basis, we have that $w(t, S_{i-1}, o_i) \leq \frac{\epsilon}{r} d$. Because of monotonicity and nonnegativity we have

$$w(t, S_{i-1}, a_i) \geq (1 - \epsilon) w(t, S_{i-1}, o_i) - \frac{\epsilon}{r} d$$

in any case.
Combining this with the Chernoff bound and using $f(O) \geq d$, we get that

$$\mathbb{E}(\Delta(a_i | R(p + \delta S_{i-1})) \geq (1 - \epsilon) \mathbb{E}(\Delta(o_i | R(p + \delta S_{i-1})) - \frac{2\epsilon}{r} f(O)$$

holds with high probability (for $n \to \infty$).
Combining everything we have so far, we can return to our telescopic sum:

$$\begin{aligned}
F(p_{t+\delta}) - F(p_t) &= \sum_{i=1}^{r} F(p_t + \delta S_i) - F(p_t + \delta S_{i-1}) \\
&\geq \sum_{i=1}^{r} \delta \mathbb{E}(\Delta(a_i | R(p + \delta S_{i-1})) \\
&\geq \sum_{i=1}^{r} \delta((1 - \epsilon) \mathbb{E}(\Delta(o_i | R(p + \delta S_{i-1})) - \frac{2\epsilon}{r} f(O)). \\
&= -2\epsilon\delta f(O)) + \delta \sum_{i=1}^{r} (1 - \epsilon) \mathbb{E}(\Delta(o_i | R(p + \delta S_{i-1})) \\
&\geq -2\epsilon\delta f(O)) + \delta \sum_{i=1}^{r} (1 - \epsilon) \mathbb{E}(\Delta(o_i | R(p + \delta S)) \\
&\geq -2\epsilon\delta f(O)) + \delta(1 - \epsilon)(F(O \cup R(p + \delta S)) - f(R(p + \delta S))) \\
&\geq -2\epsilon\delta f(O)) + \delta(1 - \epsilon)(f(O) - f(R(p + \delta S)) \\
&= \delta((1 - 3\epsilon) f(O) - f(R(p + \delta S))) = \delta((1 - 3\epsilon) f(O) - F(p_{t+\delta})),
\end{aligned}$$

where the third and fourth inequality are due to submodularity and the last is due to monotonicity of F.
Rearranging the terms, we get the equivalent formulation

$$F(p_{t+\delta}) \geq \frac{\delta(1-3\epsilon)f(O) + F(p_t)}{1+\delta}.$$

Now that we have a lower bound on the improvement in each iteration, we can apply it inductively to get

$$\begin{aligned}
F(p_t) &\geq (\frac{1}{1+\delta})^{\lfloor t/\delta \rfloor + 1} F(p_0) + \sum_{j=0}^{\lfloor t/\delta \rfloor} (\frac{1}{1+\delta})^j (\delta(1-3\epsilon)f(O)) \\
&= (\frac{1}{1+\delta})^{\lfloor t/\delta \rfloor + 1} f(\emptyset) + \delta(1-3\epsilon)f(O)) \frac{1 - (\frac{1}{1+\delta})^{\lfloor t/\delta \rfloor + 1}}{1 - \frac{1}{1+\delta}} \\
&\geq (1-3\epsilon)f(O)(1 + \delta + (\frac{1}{1+\delta})^{\lfloor t/\delta \rfloor}) \\
&\geq (1-3\epsilon)f(O)(1 + (\frac{1}{1+\delta})^{t/\delta}),
\end{aligned}$$

where we used the nonnegativity of f in the second inequality. Setting $t = 1$, we obtain the desired result:

$$\begin{aligned}
F(p_1) &\geq (1-3\epsilon)f(O)(1 + (\frac{1}{1+\delta})^{1/\delta}) \\
&\geq (1-3\epsilon)f(O)(1 + 1/e) \\
&= (1 - 1/e - O(\epsilon))f(O).
\end{aligned}$$

□

5 Conclusion

We have considered the problem of maximizing submodular functions, subject to cardinality and matroid constraints. We presented two algorithms for the cardinality constraint and showed their $(1-1/e)$- and $(1-1/e-\epsilon)$-approximation guarantees. We then modified these algorithm to work for the case of matroid constraints and proved that they achieve a $1/2$-approximation. We then discussed the continuous greedy algorithm and a modification thereof, the accelerated continuous greedy algorithm. For this last algorithm, we proved that it achieves a $(1-1/e-O(\epsilon))$-approximation and calculated its running time.

With that, we have only scratched the surface of what there is to know about submodular functions. There are many other types of constraints that can be studied, very important ones being the **knapsack contraint** and **graph contraints**. Additionally, there are other settings that can be considered, like online maximization or robust maximization. An overview of different problems can be found in [3].

References

[1] Ageev, Alexander A., and Maxim I. Sviridenko. "Pipage rounding: A new method of constructing algorithms with proven performance guarantee." Journal of Combinatorial Optimization 8 (2004): 307-328.

[2] Badanidiyuru, Ashwinkumar, and Jan Vondrák. "Fast algorithms for maximizing submodular functions." Proceedings of the twenty-fifth annual ACM-SIAM symposium on Discrete algorithms. Society for Industrial and Applied Mathematics, 2014.

[3] Krause, Andreas, and Daniel Golovin. "Submodular function maximization." Tractability 3 (2014): 71-104.

[4] Vondrák, Jan. "Optimal approximation for the submodular welfare problem in the value oracle model." Proceedings of the fortieth annual ACM symposium on Theory of computing. 2008.